

PORTADA

Técnicas de Programación

Unidad 6 — Licenciamiento, Empaquetado y Despliegue

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

11 líneas de código, retiradas de un repositorio público en unos minutos, rompieron builds de miles de proyectos en todo el mundo el mismo día.



CASO REAL

El incidente de left-pad (marzo de 2016)

Dato	Detalle
Qué era	left-pad : un paquete de npm de 11 líneas que rellenaba un string con espacios/ceros a la izquierda.
Qué pasó	Su autor, Azer Koçulu, retiró (<i>unpublish</i>) todos sus paquetes de npm tras una disputa por el nombre de otro de sus proyectos.
Impacto	Miles de proyectos —incluyendo herramientas tan usadas como Babel— dependían de left-pad de forma directa o indirecta, y sus builds empezaron a fallar en minutos en todo el mundo.
Causa raíz	Ninguno de esos proyectos había fijado ni verificado el contenido de una dependencia tan pequeña; todos confiaban en que seguiría disponible.

CASO REAL

La respuesta del ecosistema

npm restauró manualmente el paquete horas después para frenar el daño, pero el incidente expuso un problema estructural: **cualquiera puede retirar una dependencia de la que dependen miles de proyectos, en cualquier momento.**

Como consecuencia directa, npm cambió sus políticas de *unpublish* para restringir la eliminación de paquetes con muchos dependientes. El caso se volvió el ejemplo de referencia de la industria para hablar de **gestión de dependencias, empaquetado y despliegue responsable** — los tres temas de hoy.

TRANSICIÓN

Cerrando el curso: de escribir código a distribuirlo

Todo lo que has construido en el semestre —clases, interfaces, manejo de archivos, librerías de terceros— ha vivido dentro de tu propio proyecto. Hoy das el paso que falta: **convertir tu código en algo que otras personas puedan instalar, usar y confiar**, sin copiar y pegar archivos. Eso exige responder tres preguntas: ¿bajo qué condiciones dejas que lo usen (licencia)?, ¿cómo lo empaquetas?, y ¿cómo lo pones a disposición (despliegue)?

CONCEPTO

¿Qué es una licencia de software?

Una **licencia de software** es un documento legal que define qué puede y qué no puede hacer otra persona con tu código: usarlo, modificarlo, redistribuirlo, o venderlo. Sin una licencia explícita, por defecto rigen las leyes de **derechos de autor**: nadie más tiene permiso legal de usar tu código, aunque esté público en un repositorio.

Un repositorio público en GitHub sin archivo `LICENSE` no es automáticamente "libre de usar" — es, legalmente, todos los derechos reservados.

CONCEPTO

Licencias permisivas vs. copyleft

Las licencias de código abierto se agrupan en dos familias según qué exigen a quien redistribuye el código:

- **Permisivas:** permiten usar, modificar y redistribuir el código —incluso dentro de software privativo— con casi ninguna condición más allá de mantener el aviso de copyright original.
- **Copyleft:** exigen que cualquier trabajo derivado que se redistribuya se publique **bajo la misma licencia**, manteniendo el código (y sus modificaciones) abierto.



CONCEPTO

Las licencias más comunes en Java

Licencia	Tipo	Qué exige
MIT	Permisiva	Solo conservar el aviso de copyright y la licencia en las copias.
Apache 2.0	Permisiva	Igual que MIT, más una concesión explícita de patentes (usada por Commons CSV/Math, PDFBox, JFreeChart).
GPL v3	Copyleft fuerte	Todo el programa que la incluya debe distribuirse también bajo GPL, con su código fuente disponible.
LGPL v3	Copyleft débil	Solo exige abrir el código de la propia librería LGPL, no del programa que la usa como dependencia.

CASO REAL

Cisco Systems vs. Free Software Foundation (2003–2009)

Cisco distribuyó los routers Linksys WRT54G con firmware que incluía **BusyBox** y el **kernel Linux**, ambos bajo licencia **GPL**, sin publicar el código fuente modificado que la GPL exige entregar a quien recibe el binario.

La **Free Software Foundation** y luego el **Software Freedom Law Center** reclamaron el incumplimiento durante años; Cisco resistió el cambio de proceso interno hasta que en 2009 llegó a un acuerdo: publicó el código fuente exigido, nombró un responsable interno de cumplimiento de licencias open source y pagó una suma a la FSF.

No es un caso aislado ni antiguo: hoy existen equipos legales y de "compliance" dedicados solo a auditar qué licencias trae cada dependencia de un producto de software.

TRANSICIÓN

Ya elegiste la licencia — ahora, ¿cómo lo empaquetas?

Una licencia define las reglas de uso, pero no resuelve el problema técnico: ¿cómo entrega alguien su código para que otro proyecto lo agregue como dependencia, igual que agregaste Apache Commons CSV o JFreeChart en la Unidad 4? La respuesta, en el ecosistema Java, es **Maven**.

CONCEPTO

Maven: coordenadas de un artefacto

Cada dependencia que has agregado este semestre en un `pom.xml` se identifica con tres coordenadas únicas, llamadas **GAV**:

- `groupId` : quién lo publica (convención: dominio invertido, ej. `org.apache.pdfbox`).
- `artifactId` : el nombre del proyecto o librería.
- `version` : qué versión exacta se está usando.

```
<groupId>co.edu.udea</groupId>  
<artifactId>mi-libreria</artifactId>  
<version>1.0.0</version>
```



CONCEPTO

Empaquetar: de código fuente a .jar

El comando `mvn package` compila el proyecto y lo empaqueta en un archivo `.jar` (*Java ARchive*) dentro de la carpeta `target/` — el mismo formato en el que llegan las librerías que ya usas (Commons CSV, PDFBox, JFreeChart son, cada una, un `.jar`).

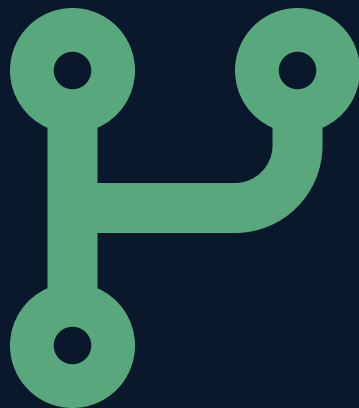
Tipo de JAR	Qué contiene
JAR normal	Solo las clases compiladas del proyecto — asume que quien lo usa ya tiene las dependencias.
Fat/Uber JAR	Las clases del proyecto más todas sus dependencias empaquetadas juntas — se ejecuta solo, sin instalar nada más.

CONCEPTO

Versionado semántico (SemVer)

Cambiar la `version` de un artefacto sin criterio rompe la confianza de quien depende de él —el problema de fondo del caso `left-pad`—. **SemVer** resuelve esto con un formato `MAYOR.MENOR.PARCHE` (ej. `2.4.1`) donde cada número comunica un significado exacto:

Número	Se incrementa cuando...
MAYOR	Hay cambios que rompen compatibilidad con versiones anteriores.
MENOR	Se agrega funcionalidad nueva, compatible con lo anterior.
PARCHE	Se corrige un bug, sin cambiar la API pública.



INTERACCIÓN

Clasifiquen el cambio de versión

En parejas, para cada cambio digan si el siguiente número de versión debe subir MAYOR, MENOR o PARCHE (partiendo de la versión actual 2.3.5):

1. Se corrige un error que hacía fallar `DescriptiveStatistics.getMean()` con listas vacías.
2. Se agrega un nuevo método `getModa()` , sin tocar los métodos existentes.
3. Se renombra el método `leerCSV()` a `leerArchivoCSV()` , eliminando el anterior.

4 minutos · justifiquen cada respuesta con la tabla de la diapositiva anterior

TRANSICIÓN

Del `.jar` en tu disco a un artefacto que otros pueden instalar

Empaquetar produce un archivo — **desplegarlo** es lo que lo pone al alcance de quien lo necesita, sin que tenga que pedirte por correo.

CONCEPTO

Dos formas de desplegar un JAR

Forma	Cuándo se usa
Publicar en un repositorio de artefactos (Maven Central, GitHub Packages)	Cuando el JAR es una librería que otros proyectos agregarán como dependencia en su propio <code>pom.xml</code> .
Distribuir un JAR ejecutable (<code>java -jar mi-app.jar</code>)	Cuando el JAR es una aplicación que se ejecuta directamente, no una dependencia de otro proyecto.



CONCEPTO

Hacer un JAR ejecutable: el manifiesto

Para que `java -jar` sepa qué clase ejecutar, el JAR necesita un archivo `META-INF/MANIFEST.MF` que declare la clase principal. En Maven, se configura declarativamente en el `pom.xml` con el plugin `maven-jar-plugin` :

```
<build>
  <plugins>
    <plugin>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>co.edu.udea.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

CONCEPTO

El checklist antes de publicar cualquier librería

1. Archivo `LICENSE` en la raíz del repositorio, con una licencia elegida a propósito (no dejarlo vacío).
2. Coordenadas Maven (`groupId` / `artifactId`) consistentes y sin conflicto con librerías existentes.
3. Número de versión que respeta SemVer, coherente con lo que realmente cambió.
4. Un `README` que explique cómo agregar la dependencia y qué hace la librería.
5. Confirmar que ninguna dependencia interna tiene una licencia **incompatible** con la que se está eligiendo (ej. no envolver código GPL dentro de un proyecto que se publica como MIT sin cumplir la GPL).

SÍNTESIS

Lo que hay que llevarse de la sesión

1. Sin licencia explícita, el código público sigue siendo "todos los derechos reservados" por defecto — MIT y Apache 2.0 son permisivas, GPL y LGPL son copyleft.
2. Elegir mal (o ignorar) la licencia tiene consecuencias legales reales: el caso Cisco/FSF terminó en una demanda y un acuerdo de cumplimiento.
3. Maven empaqueta el código en un `.jar` identificado por `groupId / artifactId / version ; SemVer ( MAYOR.MENOR.PARCHE )` comunica qué tipo de cambio trae cada versión nueva.
4. Desplegar es publicar ese artefacto (repositorio Maven) o distribuirlo como ejecutable (`java -jar` con `MANIFEST.MF`) — y el caso `left-pad` recuerda por qué la gestión responsable de dependencias importa para todo el ecosistema, no solo para tu proyecto.



CIERRE

Antes de la próxima sesión

- Revisa la licencia de al menos dos librerías que hayas usado este semestre (Commons CSV, Commons Math, JFreeChart, PDFBox, Gson) e identifica si son permisivas o copyleft.
- Piensa en el checklist de publicación de la diapositiva anterior aplicado a un proyecto propio del curso.

Próxima sesión: Sistema de Generación Automática de Reportes con IA — sigue dentro de la Unidad 6.

